Assignment-20.3

2303A51304

Batch-05

Task-01:

Prompt:

Generate a Python user registration program that accepts username and password input.

Then analyze the password validation and improve security by:

Enforcing a minimum password length.

Preventing weak passwords.

Using regular expressions to require uppercase, lowercase, numbers, and special characters.

Rejecting common passwords (e.g., 123456, password, admin).

Code:

```
n-20.2 > task-01.py > ...
import re
import getpass
# ----------- Basic User Registration -----------
def basic_registration():
    username = input("Enter username: ")
    password = getpass.getpass("Enter password: ")
    print(f"User '{username}' registered successfully with basic validation.")
    return username, password

# ----------- Improved Secure Registration -----------
def is_strong_password(password):
    if len(password) < 8:
        return False, "Password must be at least 8 characters long."
    if not re.search(r'[A-Z]', password):
        return False, "Password must contain at least one uppercase letter."
    if not re.search(r'[a-z]', password):
        return False, "Password must contain at least one lowercase letter."
    if not re.search(r'[0-9]', password):
        return False, "Password must contain at least one number."
    if not re.search(r'[@$!%*?&]', password):
        return False, "Password must contain at least one special character (@$!%*?&)."
    common_passwords = ['123456', 'password', 'admin', '12345678', 'qwerty']
    if password in common_passwords:
        return False, "Password is too common. Please choose a stronger password."
    return True, ""
def secure_registration():
    username = input("Enter username: ")
    password = getpass.getpass("Enter password: ")

    is_valid, message = is_strong_password(password)
    if is_valid:
        print(f"User '{username}' registered successfully with strong password validation.")
        return username, password
    else:
        print(f"Error: {message}")
        return None, None
# ----------- Main Program -----------
if __name__ == "__main__":
    print("Basic User Registration:")
    basic_registration()

    print("\nSecure User Registration:")
    secure_registration()
```

Output:

```
Basic User Registration:
Enter username: poojitha
Enter password:
User 'poojitha' registered successfully with basic validation.

Secure User Registration:
Enter username: poojitha
Enter password:
Error: Password must contain at least one lowercase letter.
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>
```

Explanation:

• The prompt asks AI to first create a basic Python user registration program that accepts a username and password.
• It then requires checking whether the password validation is weak, such as allowing short or common passwords.
• The AI must improve security by adding rules like minimum length and stronger validation conditions.
• Regular expressions are used to ensure the password contains uppercase letters, lowercase letters, numbers, and special characters.
• The final result should be a safer registration script along with an explanation of the security improvements made.

Task-02:

Prompt:

Generate a Python Flask file upload application.

Then analyze the program for security vulnerabilities such as:

Missing file type validation

No file size restriction

Unsafe file storage paths

Improve the program by implementing secure practices:

Allow only specific file extensions (e.g., jpg, png, pdf).

Enforce maximum file size limits.

Store files using safe and secure paths.

Prevent malicious file uploads.

Code:

n-20.2 > task-02.py > ...

```python
from flask import Flask, request, redirect, url_for, flash
import os
from werkzeug.utils import secure_filename

# ----------- Basic File Upload -----------
app = Flask(__name__)
UPLOAD_FOLDER = 'uploads'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
@app.route('/upload', methods=['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        if 'file' not in request.files:
            flash('No file part')

        file = request.files['file']
        if file.filename == '':
            flash('No selected file')
        if file:
            filename = secure_filename(file.filename)
            file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
            return 'File uploaded successfully'
    return '''
    <!doctype html>
    <title>Upload a File</title>
    <h1>Upload a File</h1>
    <form method=post enctype=multipart/form-data>
      <input type=file name=file>
      <input type=submit value=Upload>
    </form>
    '''
# ----------- Secure File Upload -----------
ALLOWED_EXTENSIONS = {'jpg', 'jpeg', 'png', 'pdf'}
MAX_CONTENT_LENGTH = 16 * 1024 * 1024  # 16 MB
app.config['MAX_CONTENT_LENGTH'] = MAX_CONTENT_LENGTH
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
@app.route('/secure_upload', methods=['GET', 'POST'])
```

```python
# ----------- Secure File Upload -----------
ALLOWED_EXTENSIONS = {'jpg', 'jpeg', 'png', 'pdf'}
MAX_CONTENT_LENGTH = 16 * 1024 * 1024  # 16 MB
app.config['MAX_CONTENT_LENGTH'] = MAX_CONTENT_LENGTH
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
@app.route('/secure_upload', methods=['GET', 'POST'])
def secure_upload_file():
    if request.method == 'POST':
        if 'file' not in request.files:
            flash('No file part')
            return redirect(request.url)
        file = request.files['file']
        if file.filename == '':
            flash('No selected file')
            return redirect(request.url)
        if file and allowed_file(file.filename):
            filename = secure_filename(file.filename)
            file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
            return 'File uploaded successfully'
        else:
            flash('Invalid file type. Allowed types: jpg, jpeg, png, pdf')
            return redirect(request.url)
    return '''
    <!doctype html>
    <title>Secure File Upload</title>
    <h1>Secure File Upload</h1>
    <form method=post enctype=multipart/form-data>
      <input type=file name=file>
      <input type=submit value=Upload>
    </form>
    '''
# ----------- Main Program -----------
if __name__ == "__main__":
    if not os.path.exists(UPLOAD_FOLDER):
        os.makedirs(UPLOAD_FOLDER)
    app.run(debug=True)
```

Output:

Explanation:

• The program creates a Flask web application that allows users to upload files through a browser interface.
• The /upload route implements a basic file upload where any file can be uploaded and saved to an uploads folder without strong security checks.
• The secure version (/secure_upload) improves safety by allowing only specific file extensions and limiting file size using Flask configuration.
• The allowed_file() function validates file types, and secure_filename() prevents unsafe file names that could harm the system.
• When the application runs, it creates the upload directory if it does not exist and starts a local web server to handle file upload requests.

Task-03:

Prompt:

Generate a Python Flask API endpoint for a simple web application (for example, a user profile or data submission API).

Then perform a security review of the generated API and identify vulnerabilities such as:

Missing authentication or authorization

Insecure or improperly used HTTP methods

Lack of rate limiting

Missing input validation

Improve the API by implementing secure practices:

Add token-based authentication (JWT or API token).

Validate and sanitize user inputs.

Apply rate limiting to prevent abuse.

Use proper HTTP methods and secure responses.

Code:

-20.2 > task-03.py > ...

```python
from flask import Flask, request, jsonify
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

app = Flask(__name__)

# Simple token authentication
API_TOKEN = "secure123token"

# Rate limiting setup
limiter = Limiter(
    get_remote_address,
    app=app,
    default_limits=["5 per minute"]
)

users = []

# -------- Authentication Function --------
def authenticate(req):
    token = req.headers.get("Authorization")
    return token == f"Bearer {API_TOKEN}"

# -------- Input Validation --------
def valid_user(data):
    if not data:
        return False
    if "name" not in data or "email" not in data:
        return False
    if not isinstance(data["name"], str):
        return False
    if "@" not in data["email"]:
        return False
    return True

# -------- Secure POST API --------
@app.route("/add-user", methods=["POST"])
```

```python
def valid_user(data):
    return True

# -------- Secure POST API --------
@app.route("/add_user", methods=["POST"])
@limiter.limit("3 per minute")
def add_user():
    if not authenticate(request):
        return jsonify({"error": "Unauthorized"}), 401

    data = request.json

    if not valid_user(data):
        return jsonify({"error": "Invalid input"}), 400

    users.append(data)
    return jsonify({"message": "User added securely"}), 201

# -------- Secure GET API --------
@app.route("/users", methods=["GET"])
@limiter.limit("5 per minute")
def get_users():
    if not authenticate(request):
        return jsonify({"error": "Unauthorized"}), 401

    return jsonify(users)

if __name__ == "__main__":
    app.run(debug=False)
```

Output:

http://127.0.0.1:5000/add_user

POST http://127.0.0.1:5000/add_user

Params Authorization Headers (11) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
  "name": "Amulya",
  "email": "amulya@email.com"
}
```

Body Cookies Headers (5) Test Results Status: 201

Pretty Raw Preview JSON

```
{"data":{"email":"amulya@email.com","name":"Amulya"},"message":"User added securely"}
```

GET http://127.0.0.1:5000/users

rams Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

dy Cookies Headers (5) Test Results

Pretty Raw Preview JSON

```
[{"email":"poojitha@email.com","name":"Poojitha"},{"email":"amulya@email.com","name":"Amulya"}]
```

Explanation:

• The program creates a Flask API server that allows users to add and view user data through HTTP endpoints.

• A fixed API token is used for authentication, and every request must include the correct Authorization header to access protected routes.

• A rate-limiting mechanism tracks requests using the client's IP address and blocks users who send too many requests within a short time.

• The valid_user() function performs input validation, ensuring only properly formatted name and email data are accepted before storing it.

• The /add_user endpoint securely adds users using POST requests, while /users retrieves stored users, and the server starts running locally when the script is executed.

Task-04:

Prompt:

Generate an HTML feedback form with JavaScript that accepts user input and displays it on the page.

Then perform a security review to check for XSS vulnerabilities, especially where user input is rendered directly without escaping.

Improve the form by:

Escaping HTML entities before displaying user input

Implementing Content Security Policy (CSP)

Preventing injection of malicious scripts

Code:

```
h-20.2 > task-04.html > html > body > h3
<!DOCTYPE html>
<html>
<head>
    <title>Feedback Form (Secure)</title>
    <!-- Content Security Policy to block inline scripts -->
    <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self';">
</head>
<body>
    <h2>Feedback Form (Secure)</h2>
    <form id="feedbackForm">
        <label>Your Feedback:</label><br>
        <input type="text" id="feedback"><br><br>
        <button type="submit">Submit</button>
    </form>

    <h3>Output:</h3>
    <div id="output"></div>

    <script>
        function escapeHTML(str) {
            return str.replace(/&/g, "&amp;")
                      .replace(/</g, "&lt;")
                      .replace(/>/g, "&gt;")
                      .replace(/"/g, "&quot;")
                      .replace(/'/g, "&#39;");
        }

        document.getElementById("feedbackForm").addEventListener("submit", function(e){
            e.preventDefault();
            var feedback = document.getElementById("feedback").value;
            // Secure: escape HTML entities
            document.getElementById("output").innerHTML = escapeHTML(feedback);
        });
    </script>
</body>
</html>
```

Output:

Feedback Form (Secure)

Your Feedback:
its was nice

Submit

Output:

Explanation:

The form takes user input through a text box and displays it on the page after submission.

In the vulnerable version, the input is inserted directly into innerHTML. This allows attackers to inject <script> tags or HTML, which executes in the browser (classic XSS).

The secure version prevents this by using a escapeHTML() function that converts special characters (<, >, &, ", ') into HTML entities, so the browser renders them as text instead of executing them.

Task-05:

Prompt:

Generate a Python program that connects to an SQLite database and fetches user details based on user input.

Then perform a security review of the code to check for SQL Injection vulnerabilities, especially cases where SQL queries are created using string concatenation.

Improve the program by:

Replacing unsafe queries with parameterized queries (prepared statements)

Validating user input

Ensuring secure database interaction

Code:

20.2 > task-03.py > ...

```python
import sqlite3
import os

DB_NAME = "users.db"

# ----------------- CREATE DATABASE AND SAMPLE DATA -----------------
def create_db():
    """Create database and table if not exists, insert sample data if table empty."""
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()

    # Create table if not exists
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT NOT NULL,
        email TEXT NOT NULL
    )
    """)

    # Check if table has data
    cursor.execute("SELECT COUNT(*) FROM users")
    count = cursor.fetchone()[0]

    if count == 0:
        # insert sample data
        sample_users = [
            ("poojitha", "poojitha@email.com"),
            ("alex", "alex@email.com"),
            ("sam", "sam@email.com")
        ]
        cursor.executemany("INSERT INTO users (username, email) VALUES (?, ?)", sample_users)
        print("Sample users added to the database.\n")
    else:
        print("Database already has users.\n")
```

```python
def create_db():
        ]
        cursor.executemany("INSERT INTO users (username, email) VALUES (?, ?)", sample_users)
        print("Sample users added to the database.\n")
    else:
        print("Database already has users.\n")

    conn.commit()
    conn.close()
# ----------------- SECURE QUERY FUNCTION -----------------
def get_user_details(username):
    """
    Fetch user details securely using parameterized query to prevent SQL injection.
    """
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()

    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (username,))
    result = cursor.fetchone()

    conn.close()
    return result
# ----------------- MAIN PROGRAM -----------------
def main():
    create_db()
    username = input("Enter username: ").strip()
    if not username:
        print("Invalid input: Username cannot be empty.")
        return
    user = get_user_details(username)
    if user:
        print("User found:", user)
    else:
        print("User not found.")

if __name__ == "__main__":
```

Output:

```
64/python.exe c:/Users/pooji/OneDrive/Desktop/AI_Assistance_coding/assign-20.2/task-05.py
Sample users added to the database.

Enter username: poojitha
User found: (1, 'poojitha', 'poojitha@email.com')
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>
```

Explanation:

The program creates or opens a database named users.db and ensures that a users table exists. If the table is empty, it inserts sample users automatically.

When you type a username, the program validates the input, making sure it’s not empty, to avoid meaningless queries.

It uses a parameterized query (? in SQLite) to fetch the user, which prevents SQL injection attacks because the input is treated as data, not SQL code.

The script fetches a single user record matching the username and prints it. If no user exists, it prints “User not found.”

The program is idempotent, meaning you can run it multiple times: it will create the table if missing, and it won’t duplicate sample users if they already exist.